

7. Packet Analysis using Wireshark: Capture and analyze network packets to understand network traffic and potential threats.

A key technical point: **the Wireshark desktop GUI cannot be directly used inside Google Colab**. The correct Colab-based approach is to use **TShark**, the command-line component of the Wireshark project, together with **Python/Scapy**. Students can generate authorized test packets, save them as a **.pcap**, analyze them using TShark, and then open the same capture file in the Wireshark desktop application if required.

Below is the **complete revised practical**, following the same structure and level as your Nmap practical. Your original practical uses Google Colab + Python + **subprocess** and a controlled target, so the same approach is retained here.

1. Definition

Packet Analysis is the process of capturing, examining, filtering, and interpreting network packets to understand communication between systems, network protocols, traffic patterns, and potential security threats.

Wireshark is an open-source network protocol analyzer used to capture and analyze network traffic. It provides detailed information about different layers of network communication, including Ethernet, IP, TCP, UDP, DNS, HTTP, ICMP, and other protocols.

For this practical, **Google Colab, Python, Scapy, and TShark** are used.

TShark is the command-line version of Wireshark and is suitable for packet analysis in Google Colab. Python and Scapy are used to generate controlled test packets, while TShark is used to analyze the resulting packet capture.

The packet capture generated in Colab can also be downloaded and opened later using the **Wireshark desktop application**.

Security Note: Packet capture and analysis must only be performed on systems, networks, and traffic for which the user has authorization. This practical uses controlled and locally generated packets for educational purposes.

2. Objectives

- Understand the concept of packet analysis.
- Understand the purpose of Wireshark.
- Understand the purpose of TShark.
- Install Wireshark command-line tools in Google Colab.

- Install Python packet-analysis libraries.
- Generate controlled network packets using Python.
- Capture packets into a PCAP file.
- Analyze packets using TShark.
- Identify Ethernet, IP, TCP, UDP, ICMP, DNS, and HTTP-related traffic.
- Identify source and destination IP addresses.
- Identify source and destination ports.
- Apply packet-analysis filters.
- Analyze TCP communication.
- Analyze UDP communication.
- Analyze ICMP communication.
- Analyze DNS communication.
- Identify potentially suspicious traffic patterns.
- Save and download packet captures for further analysis.

3. Learning Outcomes

After completing this practical, students will be able to:

- Explain packet analysis.
- Describe Wireshark and TShark.
- Explain the purpose of packet capture.
- Generate controlled network packets using Python.
- Create PCAP files using Scapy.
- Analyze PCAP files using TShark.
- Identify different network protocols.
- Identify source and destination addresses.
- Analyze TCP and UDP traffic.
- Analyze ICMP packets.
- Analyze DNS packets.
- Apply packet-analysis filters.
- Interpret packet-level information.
- Identify basic indicators of unusual network behavior.

4. Software Requirements

Requirement	Details
Platform	Google Colab
Programming Language	Python 3.x

Packet Generation	Scapy
Packet Analyzer	TShark
Network Analyzer	Wireshark/TShark
Python Library	Scapy
Python Module	subprocess
Capture Format	PCAP
Analysis Format	PCAP / PCAPNG
Internet	Required for installing packages
Target	Controlled/local test environment

5. Important Note About Wireshark and Google Colab

Google Colab provides a cloud-based notebook environment and does not provide the normal graphical desktop environment required by the Wireshark GUI.

Therefore:

Google Colab



Python



Scapy



Generate Controlled Packets



PCAP File



TShark



Packet Analysis



Wireshark Desktop (Optional)

Difference between Wireshark and TShark

Wireshark	TShark
Graphical interface	Command-line interface
Desktop application	Terminal/command-line tool
Interactive packet analysis	Scriptable packet analysis
Suitable for Windows/Linux/macOS GUI	Suitable for Google Colab
Displays packet details visually	Displays packet details as text
Uses PCAP/PCAPNG	Uses PCAP/PCAPNG

Thus, in this practical:

TShark = Wireshark packet-analysis engine used from the command line.

6. Theory

6.1 What is a Network Packet?

A **network packet** is a formatted unit of information transmitted through a computer network.

A packet may contain:

- Source address
- Destination address
- Protocol
- Port information
- Control information
- Payload

Example:

Source IP : 192.168.1.10

Destination IP : 8.8.8.8

Protocol : ICMP

6.2 What is Packet Analysis?

Packet analysis involves examining network packets to understand:

- Who is communicating?
- Who is receiving the communication?
- Which protocol is being used?
- Which ports are involved?
- How much data is transferred?
- What type of communication is occurring?
- Is there unusual traffic?

Packet analysis is widely used in:

- Network troubleshooting
- Network administration
- Cybersecurity
- Incident response
- Digital forensics
- Performance analysis

6.3 What is Wireshark?

Wireshark is an open-source network protocol analyzer.

It can be used to:

- Capture network packets
- Filter packets
- Inspect protocols
- Analyze IP communication
- Analyze TCP/UDP communication
- Examine DNS traffic
- Investigate network problems
- Support security investigations

6.4 What is TShark?

TShark is the command-line network protocol analyzer associated with Wireshark.

Example:

```
tshark -r capture.pcap
```

The **-r** option reads packets from a capture file.

TShark is particularly useful in Google Colab because it can be executed from Python using the **subprocess** module.

6.5 What is Scapy?

Scapy is a Python library used for creating, sending, receiving, manipulating, and analyzing network packets.

In this practical, Scapy is used to create **controlled educational packets** and save them into a PCAP file.

Example:

```
from scapy.all import IP, ICMP, wrpcap
```

```
packet = IP(dst="127.0.0.1")/ICMP()
```

```
wrpcap("capture.pcap", packet)
```

6.6 What is a PCAP File?

A **PCAP** file stores captured network packets.

Example:

```
packet_capture.pcap
```

The file can be analyzed using:

TShark

or:

Wireshark

7. Common Network Protocols

Protocol	Purpose
Ethernet	Local network communication
ARP	IP-to-MAC address resolution
IP	Network-layer addressing
ICMP	Network diagnostics
TCP	Reliable transport
UDP	Connectionless transport
DNS	Domain-name resolution
HTTP	Web communication
TLS	Encrypted communication
DHCP	Dynamic IP configuration

8. Common Ports

Port	Service
21	FTP
22	SSH
23	Telnet
25	SMTP
53	DNS
80	HTTP
443	HTTPS
3306	MySQL
8080	HTTP Alternate

9. Complete Google Colab Program

Students can copy the **complete program below into one Google Colab cell** and execute it.

The program:

1. Installs TShark.
2. Installs Scapy.
3. Checks TShark.
4. Generates controlled packets.
5. Creates TCP, UDP, ICMP, DNS-style test traffic.
6. Saves packets to a PCAP file.
7. Uses TShark to read the PCAP.
8. Displays packet information.
9. Applies protocol filters.
10. Displays source/destination information.
11. Performs basic traffic analysis.
12. Generates a final summary.

```
# =====
```

```
# PRACTICAL: PACKET ANALYSIS USING WIRESHARK / TSHARK
```

```
# Environment: Google Colab
```

```
# Packet Generation: Python + Scapy
```

```
# Packet Analysis: TShark
```

```
# Target: Controlled Educational Environment
```

```
# =====
```

```
# -----
```

```
# STEP 1: Install Required Software
```

```
# -----
```

```
import subprocess
```

```
import sys
```

```
import os
```

```
import pandas as pd

print("=" * 70)

print("    PACKET ANALYSIS USING WIRESHARK / TSHARK")

print("=" * 70)


print("\n[1] Installing TShark...")


install_tshark = subprocess.run(

    [

        "bash",

        "-c",

        "apt-get update -qq && "

        "DEBIAN_FRONTEND=noninteractive "

        "apt-get install -y tshark -qq"

    ],

    capture_output=True,

    text=True

)


if install_tshark.returncode == 0:

    print("TShark installation completed.")

else:

    print("TShark installation returned an error.")

    print(install_tshark.stderr)
```

```
# -----  
  
# STEP 2: Install Scapy  
  
# -----  
  
print("\n[2] Installing Scapy...")  
  
install_scapy = subprocess.run(  
    [sys.executable, "-m", "pip", "install", "-q", "scapy"],  
    capture_output=True,  
    text=True  
)  
  
if install_scapy.returncode == 0:  
    print("Scapy installation completed.")  
else:  
    print("Scapy installation error:")  
    print(install_scapy.stderr)  
  
# -----  
  
# STEP 3: Import Scapy  
  
# -----
```

```
from scapy.all import (
    IP,
    IPv6,
    TCP,
    UDP,
    ICMP,
    DNS,
    DNSQR,
    Raw,
    Ether,
    wrpcap,
    rdpcap
)

print("\n[3] Scapy imported successfully.")

# -----
# STEP 4: Check TShark Version
# -----

print("\n[4] Checking TShark installation...")

version = subprocess.run(
    ["tshark", "--version"],
```

```
capture_output=True,  
text=True  
)
```

```
if version.returncode == 0:  
    print(version.stdout.splitlines()[0])  
else:  
    print("Unable to execute TShark.")  
    print(version.stderr)
```

```
# -----
```

```
# STEP 5: Define PCAP File
```

```
# -----
```

```
pcap_file = "/content/packet_analysis_capture.pcap"
```

```
print("\n[5] PCAP File:")
```

```
print(pcap_file)
```

```
# -----
```

```
# STEP 6: Create Controlled Packets
```

```
# -----
```

```
print("\n[6] Generating controlled educational packets...")
```

```
packets = []
```

```
# -----
```

```
# ICMP PACKETS
```

```
# -----
```

```
icmp_request = (  
    IP(src="192.168.1.10", dst="8.8.8.8") /  
    ICMP(type=8) /  
    Raw(load="Educational ICMP Echo Request")  
)
```

```
icmp_reply = (  
    IP(src="8.8.8.8", dst="192.168.1.10") /  
    ICMP(type=0) /  
    Raw(load="Educational ICMP Echo Reply")  
)
```

```
packets.append icmp_request)
```

```
packets.append icmp_reply)
```

```
# -----
```

```
# TCP PACKETS
```

```
# -----
```

```
tcp_syn = (  
    IP(src="192.168.1.10", dst="192.168.1.20") /  
    TCP(sport=50000, dport=80, flags="S")  
)
```

```
tcp_syn_ack = (  
    IP(src="192.168.1.20", dst="192.168.1.10") /  
    TCP(sport=80, dport=50000, flags="SA")  
)
```

```
tcp_ack = (  
    IP(src="192.168.1.10", dst="192.168.1.20") /  
    TCP(sport=50000, dport=80, flags="A")  
)
```

```
packets.append(tcp_syn)
```

```
packets.append(tcp_syn_ack)
```

```
packets.append(tcp_ack)
```

```
# -----
```

```
# UDP PACKET
```

```
# -----
```

```
udp_packet = (  
    IP(src="192.168.1.10", dst="192.168.1.30") /  
    UDP(sport=40000, dport=5000) /  
    Raw(load="Educational UDP Data")  
)
```

```
packets.append(udp_packet)
```

```
# -----
```

```
# DNS QUERY PACKET
```

```
# -----
```

```
dns_packet = (  
    IP(src="192.168.1.10", dst="8.8.8.8") /  
    UDP(sport=53000, dport=53) /  
    DNS(  
        rd=1,  
        qd=DNSQR(qname="example.com")  
    )  
)
```

```
packets.append(dns_packet)
```

```
# -----
```

```
# HTTP-STYLE TCP PACKET
```

```
# -----
```

```
http_packet = (
```

```
    IP(src="192.168.1.10", dst="192.168.1.20") /
```

```
    TCP(sport=51000, dport=80, flags="PA") /
```

```
    Raw(
```

```
        load="GET / HTTP/1.1\r\n"
```

```
        "Host: example.com\r\n"
```

```
        "Connection: close\r\n\r\n"
```

```
    )
```

```
)
```

```
packets.append(http_packet)
```

```
# -----
```

```
# ARP PACKETS
```

```
# -----
```

```
from scapy.all import ARP
```

```
arp_request = ARP(  
    op=1,  
    psrc="192.168.1.10",  
    pdst="192.168.1.1",  
    hwsrc="02:00:00:00:00:10"  
)
```

```
arp_reply = ARP(  
    op=2,  
    psrc="192.168.1.1",  
    pdst="192.168.1.10",  
    hwsrc="02:00:00:00:00:01",  
    hwdst="02:00:00:00:00:10"  
)
```

```
packets.append(arp_request)  
packets.append(arp_reply)
```

```
# -----  
# STEP 7: Save Packets to PCAP  
# -----
```

```
print("\n[7] Saving packets to PCAP file...")
```

```
wrpcap(pcap_file, packets)
```

```
print("Packets saved successfully.")
```

```
print("Number of packets generated:", len(packets))
```

```
# -----
```

```
# STEP 8: Verify PCAP File
```

```
# -----
```

```
print("\n[8] Checking PCAP file...")
```

```
if os.path.exists(pcap_file):
```

```
    file_size = os.path.getsize(pcap_file)
```

```
    print("PCAP file exists.")
```

```
    print("File size:", file_size, "bytes")
```

```
else:
```

```
    print("PCAP file was not created.")
```

```
# -----
```

```
# STEP 9: Read PCAP Using Scapy
```

```
# -----
```

```
print("\n[9] Reading packets using Scapy...")
```

```
captured_packets = rdpcap(pcap_file)
```

```
print("Packets read:", len(captured_packets))
```

```
# -----
```

```
# STEP 10: Display Basic Packet Information
```

```
# -----
```

```
print("\n" + "=" * 70)
```

```
print("      PACKET INFORMATION")
```

```
print("=" * 70)
```

```
for i, packet in enumerate(captured_packets, start=1):
```

```
    print("\nPacket Number:", i)
```

```
    if IP in packet:
```

```
        print("Source IP   :", packet[IP].src)
```

```
        print("Destination IP :", packet[IP].dst)
```

elif ARP in packet:

```
print("Source IP   :", packet[ARP].psrc)
```

```
print("Destination IP :", packet[ARP].pdst)
```

```
print("Protocol   :", packet.summary())
```

```
# -----
```

```
# STEP 11: Analyze All Packets Using TShark
```

```
# -----
```

```
print("\n" + "=" * 70)
```

```
print("          TSHARK PACKET ANALYSIS")
```

```
print("=" * 70)
```

```
tshark_all = subprocess.run(
```

```
[
```

```
    "tshark",
```

```
    "-r",
```

```
    pcap_file,
```

```
    "-T",
```

```
    "fields",
```

```
    "-E",
```

```
    "header=y",
```

```
"-E",  
  
"separator=",  
  
,  
  
"-e",  
  
"frame.number",  
  
"-e",  
  
"ip.src",  
  
"-e",  
  
"ip.dst",  
  
"-e",  
  
"_ws.col.Protocol",  
  
"-e",  
  
"frame.len"  
  
],  
  
capture_output=True,  
  
text=True  
  
)
```

```
print(tshark_all.stdout)
```

```
# -----
```

```
# STEP 12: Analyze TCP Packets
```

```
# -----
```

```
print("\n" + "=" * 70)
```

```
print("          TCP PACKET ANALYSIS")
```

```
print("=" * 70)
```

```
tcp_analysis = subprocess.run(
```

```
[
```

```
    "tshark",
```

```
    "-r",
```

```
    pcap_file,
```

```
    "-Y",
```

```
    "tcp",
```

```
    "-T",
```

```
    "fields",
```

```
    "-E",
```

```
    "header=y",
```

```
    "-E",
```

```
    "separator=",
```

```
    "-e",
```

```
    "frame.number",
```

```
    "-e",
```

```
    "ip.src",
```

```
    "-e",
```

```
    "ip.dst",
```

```
    "-e",
```

```
    "tcp.srcport",
```

```
        "-e",
        "tcp.dstport",
        "-e",
        "tcp.flags.str"
    ],
    capture_output=True,
    text=True
)
```

```
print(tcp_analysis.stdout)
```

```
# -----
# STEP 13: Analyze UDP Packets
# -----
```

```
print("\n" + "=" * 70)
print("          UDP PACKET ANALYSIS")
print("=" * 70)
```

```
udp_analysis = subprocess.run(
    [
        "tshark",
        "-r",
        pcap_file,
```

```
"-Y",  
  
"udp",  
  
"-T",  
  
"fields",  
  
"-E",  
  
"header=y",  
  
"-E",  
  
"separator=",  
  
"-e",  
  
"frame.number",  
  
"-e",  
  
"ip.src",  
  
"-e",  
  
"ip.dst",  
  
"-e",  
  
"udp.srcport",  
  
"-e",  
  
"udp.dstport"  
  
],  
  
capture_output=True,  
  
text=True  
  
)  
  
print(udp_analysis.stdout)
```

```
# -----  
# STEP 14: Analyze ICMP Packets  
# -----
```

```
print("\n" + "=" * 70)
```

```
print("          ICMP PACKET ANALYSIS")
```

```
print("=" * 70)
```

```
icmp_analysis = subprocess.run(  
    [  
        "tshark",  
        "-r",  
        pcap_file,  
        "-Y",  
        "icmp",  
        "-T",  
        "fields",  
        "-E",  
        "header=y",  
        "-E",  
        "separator=",  
        "-e",  
        "frame.number",  
        "-e",
```

```

        "ip.src",
        "-e",
        "ip.dst",
        "-e",
        "icmp.type",
        "-e",
        "icmp.code"
    ],
    capture_output=True,
    text=True
)

print(icmp_analysis.stdout)

```

```

# -----
# STEP 15: Analyze DNS Packets
# -----

```

```

print("\n" + "=" * 70)

print("          DNS PACKET ANALYSIS")

print("=" * 70)

```

```

dns_analysis = subprocess.run(
    [

```

```
"tshark",  
"-r",  
pcap_file,  
"-Y",  
"dns",  
"-T",  
"fields",  
"-E",  
"header=y",  
"-E",  
"separator=",  
"-e",  
"frame.number",  
"-e",  
"ip.src",  
"-e",  
"ip.dst",  
"-e",  
"dns.qry.name",  
"-e",  
"dns.qry.type"  
],  
capture_output=True,  
text=True  
)
```

```
print(dns_analysis.stdout)
```

```
# -----
```

```
# STEP 16: Analyze ARP Packets
```

```
# -----
```

```
print("\n" + "=" * 70)
```

```
print("          ARP PACKET ANALYSIS")
```

```
print("=" * 70)
```

```
arp_analysis = subprocess.run(
```

```
[
```

```
    "tshark",
```

```
    "-r",
```

```
    pcap_file,
```

```
    "-Y",
```

```
    "arp",
```

```
    "-T",
```

```
    "fields",
```

```
    "-E",
```

```
    "header=y",
```

```
    "-E",
```

```
    "separator=",
```

```
"-e",  
  
"frame.number",  
  
"-e",  
  
"arp.src.proto_ipv4",  
  
"-e",  
  
"arp.dst.proto_ipv4",  
  
"-e",  
  
"arp.opcode"  
  
],  
  
capture_output=True,  
  
text=True  
  
)
```

```
print(arp_analysis.stdout)
```

```
# -----
```

```
# STEP 17: Analyze HTTP Packets
```

```
# -----
```

```
print("\n" + "=" * 70)
```

```
print("          HTTP PACKET ANALYSIS")
```

```
print("=" * 70)
```

```
http_analysis = subprocess.run(
```

```
[  
    "tshark",  
    "-r",  
    pcap_file,  
    "-Y",  
    "http",  
    "-T",  
    "fields",  
    "-E",  
    "header=y",  
    "-E",  
    "separator=",  
    "-e",  
    "frame.number",  
    "-e",  
    "ip.src",  
    "-e",  
    "ip.dst",  
    "-e",  
    "http.request.method",  
    "-e",  
    "http.host",  
    "-e",  
    "http.request.uri"  
],
```

```
capture_output=True,  
text=True  
)
```

```
print(http_analysis.stdout)
```

```
# -----  
# STEP 18: Identify TCP SYN Packets  
# -----
```

```
print("\n" + "=" * 70)  
print("      TCP SYN ANALYSIS")  
print("=" * 70)
```

```
syn_analysis = subprocess.run(  
    [  
        "tshark",  
        "-r",  
        pcap_file,  
        "-Y",  
        "tcp.flags.syn == 1",  
        "-T",  
        "fields",  
        "-E",
```

```
"header=y",  
"-E",  
"separator=",  
"-e",  
"frame.number",  
"-e",  
"ip.src",  
"-e",  
"ip.dst",  
"-e",  
"tcp.srcport",  
"-e",  
"tcp.dstport",  
"-e",  
"tcp.flags.str"  
],  
capture_output=True,  
text=True  
)
```

```
print(syn_analysis.stdout)
```

```
# -----
```

```
# STEP 19: Protocol Statistics
```

```
# -----
```

```
print("\n" + "=" * 70)
```

```
print("          PROTOCOL STATISTICS")
```

```
print("=" * 70)
```

```
protocol_stats = subprocess.run(
```

```
[
```

```
    "tshark",
```

```
    "-r",
```

```
    pcap_file,
```

```
    "-q",
```

```
    "-z",
```

```
    "io,phs"
```

```
],
```

```
capture_output=True,
```

```
text=True
```

```
)
```

```
print(protocol_stats.stdout)
```

```
# -----
```

```
# STEP 20: Packet Count
```

```
# -----
```

```

print("\n" + "=" * 70)

print("          PACKET SUMMARY")

print("=" * 70)


print("Total Packets Generated :", len(packets))

print("PCAP File          :", pcap_file)

print("PCAP Size           :", os.path.getsize(pcap_file), "bytes")


# -----

# STEP 21: Completion Message

# -----


print("\n" + "=" * 70)

print("          PACKET ANALYSIS COMPLETED")

print("=" * 70)


print("\nStudents should record the actual TShark output")

print("and complete the observation tables.")

```

10. Program Explanation

Step 1: Import Python Libraries

```

import subprocess

import sys

```

```
import os
```

```
import pandas as pd
```

subprocess

The **subprocess** module allows Python to execute operating-system commands.

It is used to execute:

```
apt-get
```

```
tshark
```

sys

The **sys** module provides information about the Python environment.

It is used to identify the current Python executable:

```
sys.executable
```

This allows Scapy to be installed into the current Colab Python environment.

os

The **os** module is used to perform operating-system-related operations.

For example:

```
os.path.exists()
```

checks whether the PCAP file exists.

11. Installing TShark

The program executes:

```
apt-get update
```

followed by:

```
apt-get install -y tshark
```

TShark is the command-line packet analyzer used for packet analysis in Google Colab.

12. Installing Scapy

The program executes:

```
python -m pip install scapy
```

Scapy provides Python functionality for constructing and manipulating network packets.

13. Checking TShark Installation

The program executes:

```
tshark --version
```

This verifies that TShark is installed correctly.

14. Creating the PCAP File

The program defines:

```
pcap_file = "/content/packet_analysis_capture.pcap"
```

Google Colab stores the generated file in the `/content` directory.

15. Creating ICMP Packets

The program creates an ICMP Echo Request:

```
IP(src="192.168.1.10", dst="8.8.8.8") / ICMP(type=8)
```

Here:

- `src` = source IP
- `dst` = destination IP
- `type=8` = ICMP Echo Request

An ICMP Echo Reply uses:

```
ICMP(type=0)
```

16. Creating TCP Packets

The program creates a TCP SYN packet:

```
TCP(sport=50000, dport=80, flags="S")
```

Here:

- `sport=50000` → source port
- `dport=80` → destination port
- `flags="S"` → SYN

The TCP handshake is represented using:

SYN

↓

SYN/ACK

↓

ACK

17. Creating UDP Packet

The program creates:

```
UDP(sport=40000, dport=5000)
```

UDP is connectionless and does not use the TCP three-way handshake.

18. Creating DNS Packet

The program creates a DNS query:

```
DNS(  
    rd=1,  
    qd=DNSQR(qname="example.com")  
)
```

The query requests information about:

example.com

The DNS packet uses:

UDP source port → 53000

UDP destination port → 53

19. Creating HTTP-Style Packet

The program creates an HTTP request:

GET / HTTP/1.1

Host: example.com

The packet is carried over TCP port 80.

This is a **controlled educational packet**, not a live request to the website.

20. Creating ARP Packets

The program creates:

ARP(op=1)

for an ARP request.

An ARP reply uses:

ARP(op=2)

Therefore:

1 → ARP Request

2 → ARP Reply

21. Saving Packets

The following statement saves the generated packets:

```
wrpcap(pcap_file, packets)
```

The resulting file is:

```
packet_analysis_capture.pcap
```

22. Reading PCAP with Scapy

The file can be reopened using:

```
captured_packets = rdpcap(pcap_file)
```

The number of packets is then displayed.

23. TShark Packet Analysis

The basic command is:

```
tshark -r packet_analysis_capture.pcap
```

The `-r` option means:

Read packets from a capture file.

24. TShark Field Extraction

The program uses:

`-T fields`

to extract individual packet fields.

For example:

`-e ip.src`

extracts the source IP address.

`-e ip.dst`

extracts the destination IP address.

`-e tcp.srcport`

extracts the TCP source port.

`-e tcp.dstport`

extracts the TCP destination port.

25. TCP Filter

The program uses:

-Y tcp

This is a TShark display filter that displays TCP packets.

Equivalent Wireshark display filter:

tcp

26. UDP Filter

-Y udp

Equivalent Wireshark filter:

udp

27. ICMP Filter

-Y icmp

Equivalent Wireshark filter:

icmp

28. DNS Filter

-Y dns

Equivalent Wireshark filter:

dns

29. ARP Filter

-Y arp

Equivalent Wireshark filter:

arp

30. HTTP Filter

-Y http

Equivalent Wireshark filter:

http

31. TCP SYN Analysis

The program uses:

```
tcp.flags.syn == 1
```

This identifies TCP packets with the SYN flag set.

Students can use this information to understand TCP connection establishment and investigate repeated connection attempts in a real authorized capture.

32. Understanding TShark Output

A typical output can contain:

```
frame.number,ip.src,ip.dst,protocol,frame.len
```

Example:

```
1,192.168.1.10,8.8.8.8,ICMP,74
```

```
2,8.8.8.8,192.168.1.10,ICMP,74
```

Interpretation:

Field	Meaning
Frame Number	Packet sequence number
IP Source	Sender IP
IP Destination	Receiver IP
Protocol	Protocol used
Frame Length	Packet size

33. Understanding TCP Output

Example:

```
1,192.168.1.10,192.168.1.20,50000,80,SYN
2,192.168.1.20,192.168.1.10,80,50000,SYN,ACK
3,192.168.1.10,192.168.1.20,50000,80,ACK
```

This represents the TCP three-way handshake.

Client

Server

SYN ----->

<----- SYN/ACK

ACK ----->

34. Understanding DNS Output

Example:

Frame	Source	Destination	Query
6	192.168.1.10	8.8.8.8	example.com

This indicates that the source generated a DNS query for:

example.com

35. Understanding ICMP Output

Example:

Source	Destination	Type
192.168.1.10	8.8.8.8	8

8.8.8.8 192.168.1.10 0

Where:

8 = Echo Request

0 = Echo Reply

36. Understanding ARP Output

Example:

Source IP Destination IP

192.168.1.10 192.168.1.1

This represents an ARP request in the controlled capture.

37. Wireshark Display Filters

Although the practical runs TShark in Google Colab, students should learn the equivalent Wireshark filters.

Purpose	Wireshark Filter
TCP	<code>tcp</code>
UDP	<code>udp</code>
ICMP	<code>icmp</code>
DNS	<code>dns</code>
ARP	<code>arp</code>

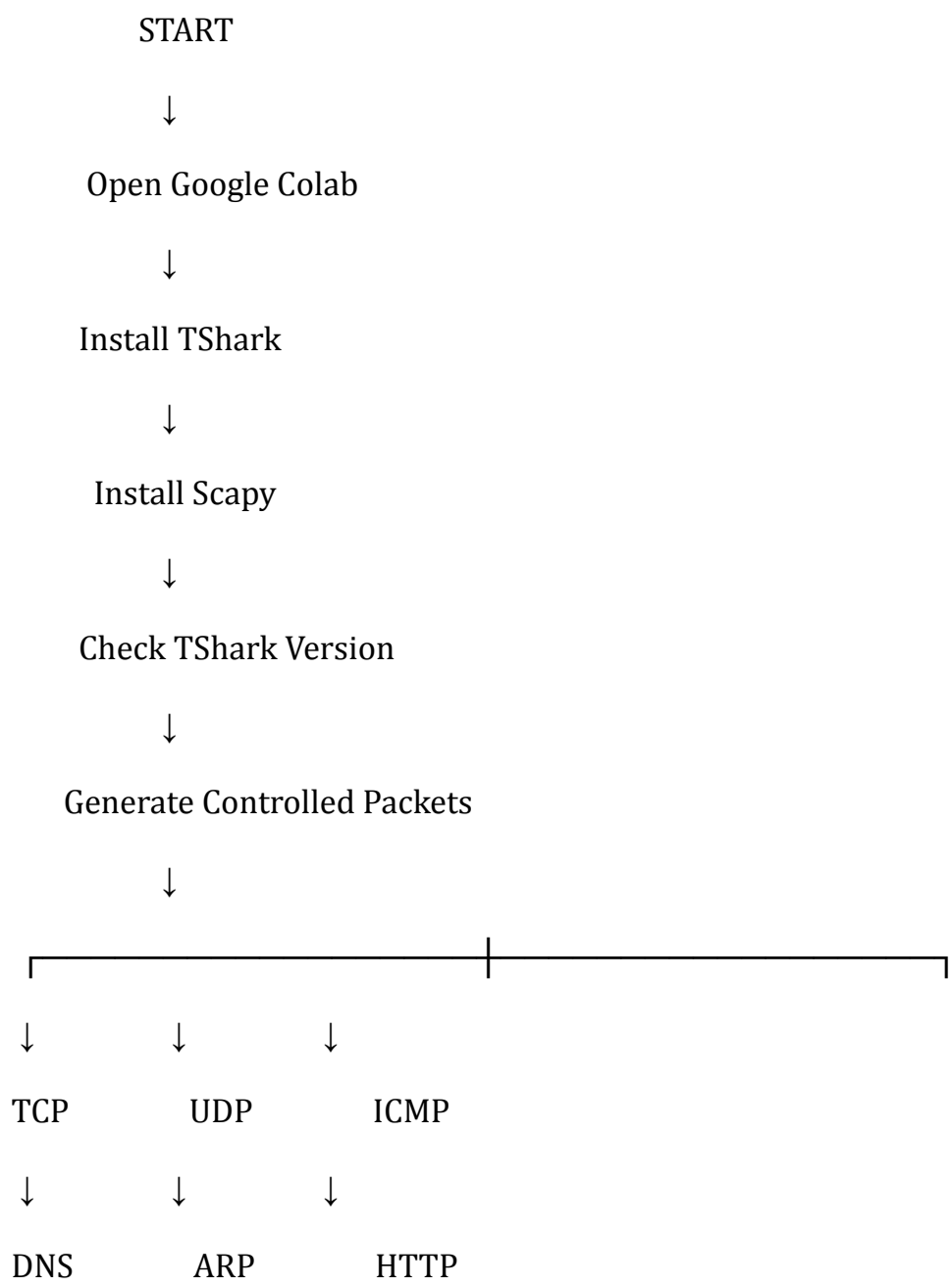
HTTP	<code>http</code>
TLS	<code>tls</code>
Specific IP	<code>ip.addr == 192.168.1.10</code>
Source IP	<code>ip.src == 192.168.1.10</code>
Destination IP	<code>ip.dst == 8.8.8.8</code>
TCP Port	<code>tcp.port == 80</code>
UDP Port	<code>udp.port == 53</code>
SYN packets	<code>tcp.flags.syn == 1</code>
SYN without ACK	<code>tcp.flags.syn == 1 && tcp.flags.ack == 0</code>

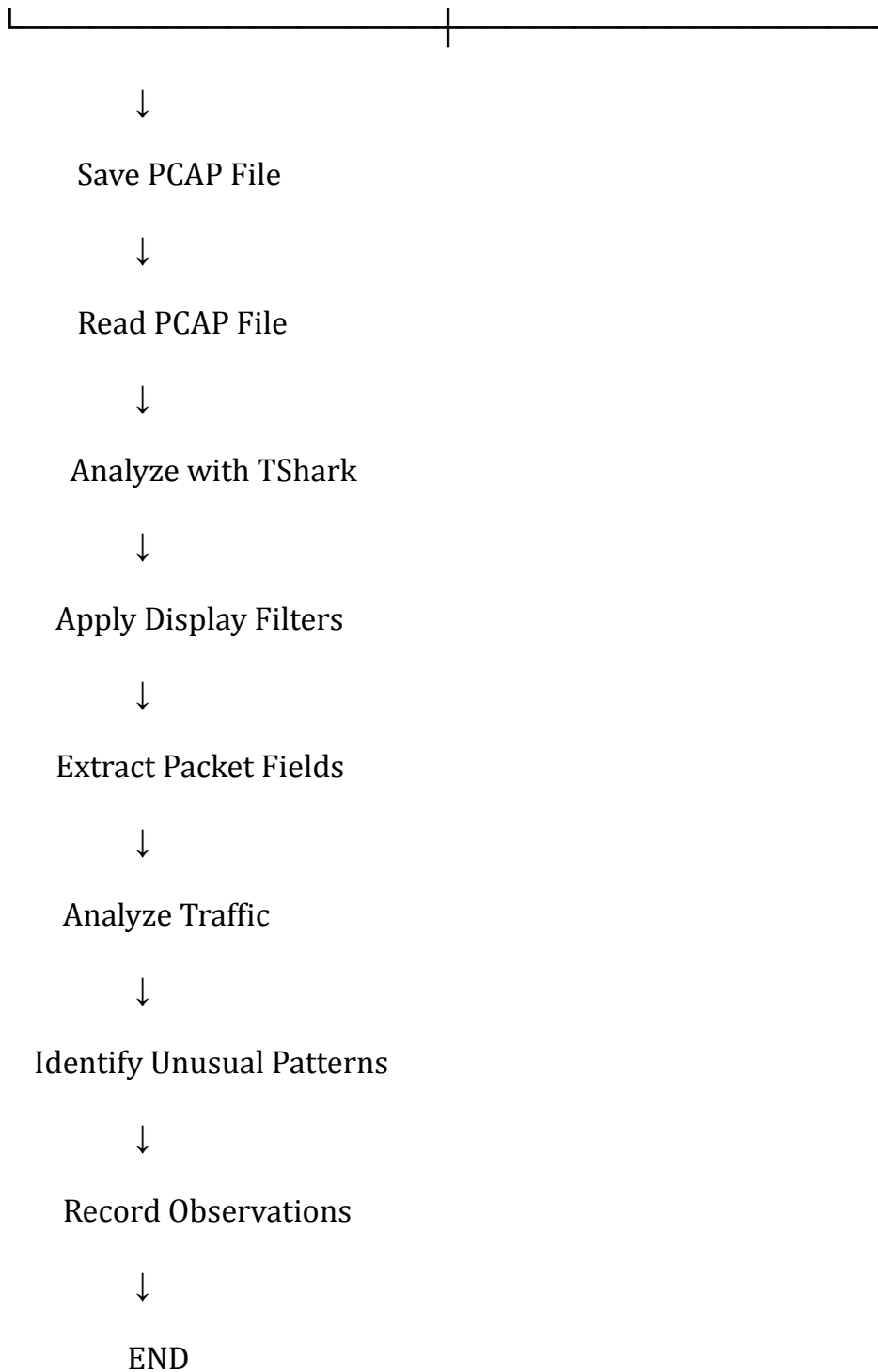
38. TShark Commands Used in Google Colab

Command	Purpose
<code>tshark --version</code>	Check TShark
<code>tshark -r file.pcap</code>	Read PCAP
<code>tshark -r file.pcap -Y tcp</code>	Analyze TCP
<code>tshark -r file.pcap -Y udp</code>	Analyze UDP

<code>tshark -r file.pcap -Y dns</code>	Analyze DNS
<code>tshark -r file.pcap -Y icmp</code>	Analyze ICMP
<code>tshark -r file.pcap -Y arp</code>	Analyze ARP
<code>tshark -r file.pcap -Y http</code>	Analyze HTTP
<code>tshark -r file.pcap -T fields</code>	Extract fields
<code>tshark -r file.pcap -q -z io,phs</code>	Protocol statistics

39. Execution Flow





40. Algorithm

Algorithm: Packet Analysis Using Wireshark/TShark

1. Start.
2. Open Google Colab.
3. Install TShark.
4. Install Scapy.
5. Verify TShark installation.
6. Import the required Python libraries.

7. Create controlled ICMP packets.
8. Create controlled TCP packets.
9. Create controlled UDP packets.
10. Create a DNS query packet.
11. Create an HTTP-style packet.
12. Create controlled ARP packets.
13. Store all packets in a packet list.
14. Save packets into a PCAP file.
15. Read the PCAP file using Scapy.
16. Read the PCAP file using TShark.
17. Apply TCP filter.
18. Apply UDP filter.
19. Apply ICMP filter.
20. Apply DNS filter.
21. Apply ARP filter.
22. Apply HTTP filter.
23. Analyze TCP SYN packets.
24. Extract source and destination information.
25. Analyze protocol statistics.
26. Record observations.
27. Save the PCAP file.
28. Stop.

41. Observation Table

Students should record the actual output generated by Google Colab.

Sr. No.	Source IP	Destination IP	Protocol	Source Port	Destination Port	Packet Length
1	_____	_____	_____	_____	_____	_____
2	_____	_____	_____	_____	_____	_____
3	_____	_____	_____	_____	_____	_____

4	_____	_____	_____	_____	_____	_____
5	_____	_____	_____	_____	_____	_____
6	_____	_____	_____	_____	_____	_____
7	_____	_____	_____	_____	_____	_____
8	_____	_____	_____	_____	_____	_____

42. Protocol Observation

Sr. No.	Protocol	Filter	Packets Observed	Observation
1	TCP	tcp	_____	_____
2	UDP	udp	_____	_____
3	ICMP	icmp	_____	_____
4	DNS	dns	_____	_____
5	ARP	arp	_____	_____
6	HTTP	http	_____	_____

43. TCP Analysis Table

Sr. No.	Source IP	Destination IP	Source Port	Destination Port	TCP Flag
1	_____	_____	_____	_____	_____
2	_____	_____	_____	_____	_____
3	_____	_____	_____	_____	_____
4	_____	_____	_____	_____	_____

44. DNS Analysis Table

Sr. No.	Source IP	Destination IP	Domain Query	Query Type
1	_____	_____	_____	_____
2	_____	_____	_____	_____

45. ICMP Analysis Table

Sr. No.	Source IP	Destination IP	ICMP Type	Meaning
1	_____	_____	_____	_____
2	_____	_____	_____	_____

46. Security Analysis

Students should analyze whether the captured traffic contains indicators requiring further investigation.

Security Indicator	Observation
Unexpected source IP	_____
Unexpected destination IP	_____
Unusual port	_____
Repeated SYN packets	_____
Excessive DNS requests	_____
Unknown protocol	_____
Unusual packet size	_____

Unusual traffic pattern	_____
-------------------------	-------

Important Interpretation

Students must understand that:

An unusual packet does not automatically mean that an attack has occurred.

Packet analysis provides evidence that should be interpreted along with system logs, application behavior, firewall logs, and other security information.

47. Export PCAP from Google Colab

After generating the PCAP file, students can download it using:

from google.colab import files

```
files.download("/content/packet_analysis_capture.pcap")
```

After downloading, students can open the file using the **Wireshark desktop application**.

Google Colab



packet_analysis_capture.pcap



Download



Open with Wireshark



Graphical Packet Analysis

48. Optional: Open the PCAP with Wireshark

After downloading:

1. Install Wireshark on the local computer.
2. Open Wireshark.
3. Select **File** → **Open**.
4. Select:

packet_analysis_capture.pcap

5. Apply filters such as:

tcp

udp

dns

icmp

arp

http